Table of Contents

1. Introduction

In the era of **Automation** and **Smart Infrastructure**, integrating robotics into daily life has become a necessity. This project introduces a synchronized system consisting of an **Autonomous Garage Gate** and a **Bluetooth-controlled 4WD Vehicle**. The system utilizes **Embedded Systems** and **Wireless Communication** to create a seamless user experience, minimizing manual intervention and maximizing efficiency.

---

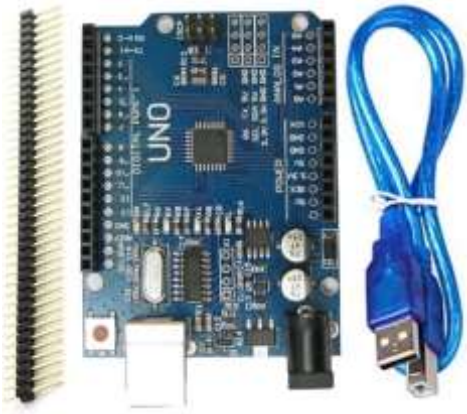
2. System Objectives

- **Safety & Automation:** Automatically detecting approaching vehicles to operate the garage doors.
- **Wireless Control:** Using **Bluetooth Technology** for stable, long-range vehicle maneuverability.
- **Voice Integration:** Implementing **Speech Recognition** to provide hands-free operation.
- **High Torque Mobility:** Utilizing a **4WD (Four-Wheel Drive)** system for robust movement on different terrains.

3. Detailed Hardware Specifications

The system is built on a modular architecture, using the following components:

- **Microcontroller (Arduino Uno):** The primary controller responsible for processing sensor data and executing commands.



- **HC-SR04 Ultrasonic Sensor:** Uses sound waves to measure distance, providing the input for the garage door automation.

- **L298N Motor Driver:** A dual H-bridge module that allows the Arduino to control the direction and speed of high-power DC motors.

- **4x DC Gear Motors:** Provide high torque and reliable traction for the vehicle's movement.

- **2x SG90 Servo Motors:** Precision actuators that control the mechanical opening and closing of the garage gate.

- **HC-05 Bluetooth Module:** Facilitates serial communication between the Arduino and the mobile application.

- **Power Supply: 3x 18650 Lithium-ion batteries** ensure a steady 12V supply for the motor driver and 5V for the logic circuit.

4. Software Design & Logic

4.1. The Autonomous Gate (Smart_garage.ino)

The logic relies on **Real-time Proximity Detection**. The sensor calculates distance using the speed of sound.

- **Trigger Distance:** 20 cm.
- **Operation:** When the condition is met, `Servo1` rotates to 0° and `Servo2` to 180°.
- **Hold Time:** A `delay(4000)` is implemented to keep the door open for 4 seconds before resetting to the closed position (90° and 80°).

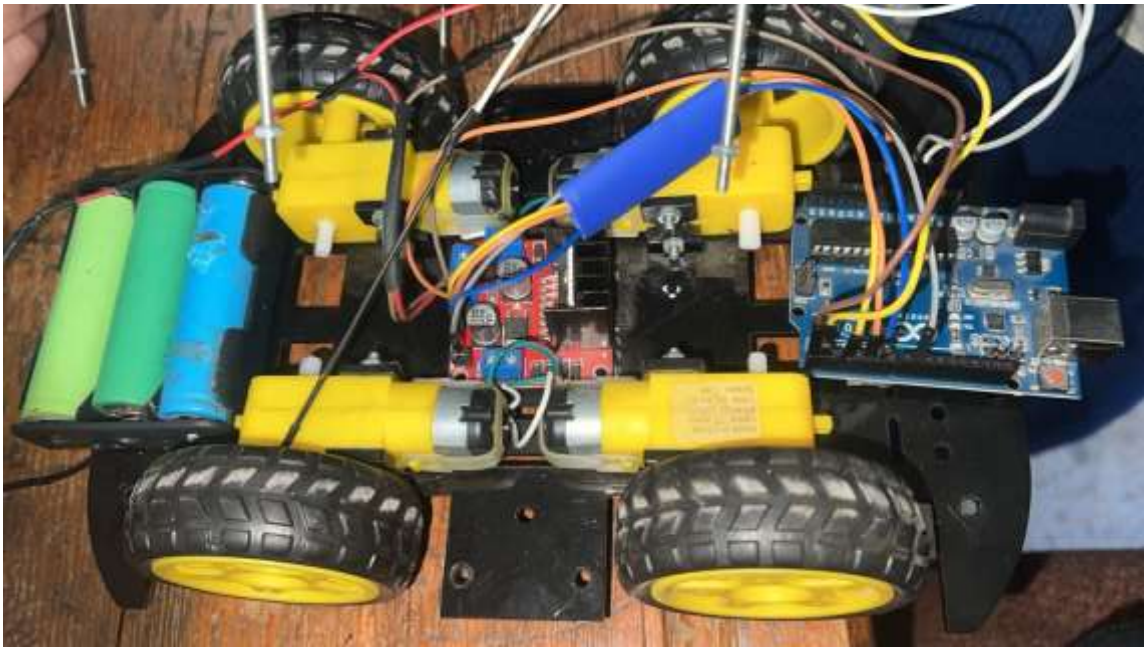
4.2. Vehicle Control Logic (RC.ino)

The vehicle uses a **Switch Case** mechanism to process commands received via the **Serial Port**.

- **Command Set:**
  - `Case 1`: Forward (All motors high).
  - `Case 2`: Backward (Reverse polarity).
  - `Case 3/4`: Turning (Differential steering).
  - `Case 5`: Stop (All pins low).

5. Circuit Connectivity & Pin Mapping

Component	Arduino Pin	Function
Ultrasonic Trig	Pin 5	Signal Output
Ultrasonic Echo	Pin 6	Signal Input
Servo 1	Pin 2	Gate Control
Servo 2	Pin 3	Gate Control
L298N Inputs	Pins 3, 5, 6, 9	**PWM** Movement Control
Bluetooth TX/RX	Hardware Serial	Wireless Data



6. Mobile Application & UI Design

The application was developed using **MIT App Inventor**. It features a modern interface with:

- **Interactive Control Pad:** Using `TouchDown` and `TouchUp` logic for instant response.
- **Voice Control Module:** Translating speech into commands via Google's API.
- **Status Monitor:** Displays the connection status with the HC-05 module.

---

7. Computer Organization Perspective

To align with the principles of **Computer Organization**, this project demonstrates how a **Central Processing Unit (CPU)** manages peripheral devices and data flow:

- **Instruction Execution Cycle:** The **ATmega328P** microcontroller follows a **Fetch-Decode-Execute** cycle to process data. It **fetches** distance data from the sensor, **decodes** the logic, and **executes** the output by sending signals to the Servo motors.
- **Input/Output (I/O) Mapping:** All hardware components are mapped to specific **Digital and PWM Pins**. This represents how a CPU communicates with external devices through a defined **Address Space**.

- **Data Communication Protocol:** The project utilizes the **UART (Universal Asynchronous Receiver-Transmitter)** protocol for communication between the **HC-05 Bluetooth Module** and the Arduino. Data is transmitted serially at a **Baud Rate** of 9600 bps.
- **PWM (Pulse Width Modulation) Control:** To control the motor speed via the **L298N**, the system uses **PWM** to simulate an analog voltage by varying the **Duty Cycle** of a digital signal.
- **Register-Level Interaction:** The mobile app sends **Byte Numbers** (e.g., 1, 2, 3) which are stored in the Arduino's **Data Registers** and processed using a **Switch-Case** mechanism for high-speed decision-making
- .

Data Mapping Table

Arduino Action ((Output	Transmitted Byte ((Data Bus	Voice/Button Logic	User Command ((Input
All Motors High	**1**	Forward" / Arrow " Up	**Move Forward**
All Motors Reverse	**2**	Backward" / " Arrow Down	**Move Backward**
Differential Steering	**3**	Right" / Arrow " Right	**Turn Right**
Differential Steering	**4**	Left" / Arrow " Left	**Turn Left**
All Motors Low	**5**	Stop" / Button " Release	**Full Stop**

System Architecture: Data Flow & Control Path

The following analysis details the end-to-end propagation of a command within the system, tracing the signal from the user's physical input to the vehicle's kinetic output. The process is divided into four distinct operational domains:

1. User Interface Domain (Input Generation)

The control sequence initiates at the Mobile Application layer, developed using MIT App Inventor.

- **Touch Interaction:** The application utilizes an "Interactive Control Pad" logic. When the user touches a directional arrow (e.g., Forward), the application registers a TouchDown event.
- **Command Encoding:** This event triggers the generation of a specific 1-byte integer command (e.g., sending the ASCII value 1 for Forward).
- **Latency Management:** Upon releasing the button, a TouchUp event immediately sends the command 5 (Stop), ensuring instant cessation of movement and preventing run-away errors.

2. Wireless Communication Domain (Transmission)

Data is transmitted wirelessly to bridge the gap between the controller (smartphone) and the vehicle.

- **Transmission:** The smartphone's internal Bluetooth radio modulates the digital command onto a 2.4 GHz carrier wave.
- **Reception:** The **HC-05 Bluetooth Module** receives the RF signal and demodulates it back into a digital TTL voltage signal.
- **Protocol (UART):** Communication follows the **Universal Asynchronous Receiver-Transmitter (UART)** protocol. The system is synchronized at a **Baud Rate of 9600 bps** to ensure data integrity between the transmitter and receiver.

3. Embedded Processing Domain (Logic & Decision)

The **Arduino Uno** acts as the central processing unit, executing the **Fetch-Decode-Execute** cycle.

- **Buffering:** The incoming byte travels from the HC-05 TX pin to the Arduino RX pin and is stored in the **Serial Receive Buffer**. This prevents data loss if the processor is temporarily busy.
- **Fetching:** The firmware's loop() function polls the buffer using Serial.available() and retrieves the byte using Serial.read().

- **Decoding:** A **Switch-Case** mechanism compares the retrieved byte against the pre-defined **Data Mapping Table** (Cases 1 through 5). This method optimizes the Program Counter (PC) path for faster execution compared to nested if-else structures.

4. Power & Actuation Domain (Execution)

The final stage converts low-voltage logic signals into high-power mechanical force.

- **Signal Output:** Based on the decoded case (e.g., Case 1: Forward), the Arduino sends digital signals to the mapped I/O pins (Pins 3, 5, 6, 9).
- **Power Amplification:** The **L298N Motor Driver** receives these low-current logic signals. It acts as a dual H-Bridge, switching the high-current path from the 12V Li-ion battery pack to the motors.
- **Electromechanical Conversion:** The **DC Gear Motors** receive the amplified current. The interaction between the stator's magnetic field and the rotor's current generates torque, rotating the wheels and achieving the desired physical movement.

4.1 Smart Garage Autonomous Control Algorithm

This section details the software logic controlling the autonomous entry system. The implementation relies on a continuous feedback loop that monitors environmental data via ultrasonic waves and executes mechanical actuation based on proximity thresholds.

4.1.1 Hardware-Software Interface

The system interacts with the physical components through the following Pin Mapping configuration, defined in the setup() function:

Component	Variable Name	Arduino Pin	Mode	Description
Servo Motor 1	servo1	Pin 2	PWM Output	Controls Left Gate Leaf
Servo Motor 2	servo2	Pin 3	PWM	Controls Right Gate

Component	Variable Name	Arduino Pin	Mode	Description
			Output	Leaf
Ultrasonic Trigger	trigPin	Pin 5	Digital Output	Initiates Sound Pulse
Ultrasonic Echo	echoPin	Pin 6	Digital Input	Receives Reflected Signal

4.1.2 Operational Logic Cycle

The loop() function executes a repetitive **Sense-Process-Act** cycle, designed to detect approaching vehicles in real-time.

1. Data Acquisition (Sensing Phase) To initiate a measurement, the controller sends a precise 10-microsecond High pulse to the trigPin. This triggers the **HC-SR04** sensor to emit an 8-cycle sonic burst at 40 kHz. The system then listens on the echoPin using the pulseIn() function, which measures the duration (in microseconds) for the sound wave to travel to the obstacle and return.

2. Signal Processing (Calculation Phase) The raw time duration is converted into physical distance using the speed of sound constant (340m/s or 0.034 cm/microsecond). The formula utilized in the code is:

Distance(cm) = (Duration * 0.034) / 2

3. Decision & Actuation (Control Phase) The logic implements a conditional threshold to automate the gate operation:

- **Condition:** If Distance <= 20 cm
- **Action (Open State):** The system rotates servo1 to and servo2 to . The opposing angles are necessary due to the mirrored physical mounting of the motors on the gate pillars.

- **Hysteresis (Safety Hold):** A delay(4000) instruction effectively locks the system in the "Open" state for 4 seconds. This safety mechanism ensures the vehicle has sufficient time to clear the entrance before the logic resumes.
- **Default State (Closed):** If the condition is not met (Distance cm), the system actively holds the gates at the closed positions (and). The value for servo2 was derived through calibration to ensure proper mechanical alignment without motor stalling.

4.1.3 Initialization

Upon startup, the setup() routine initializes the Serial Monitor at **9600 baud** for debugging purposes and immediately drives the servos to the closed position. This "Safety-First" initialization prevents erratic gate movement during the microcontroller's power-on sequence.

8. Advanced System Management (Memory & Timing)

In this section, we analyze the efficiency of the system architecture from a low-level perspective:

- **Instruction Latency & ALU Processing:** By transmitting data in **1-Byte** format, we minimized the **Instruction Latency**. This ensures that the **Arithmetic Logic Unit (ALU)** of the ATmega328P processes commands with minimal clock cycles, resulting in a near-instantaneous vehicle response.
- **Communication Timing (Polling Method):** The system implements a **Polling Mechanism** using the **Clock1 Timer** component. This allows the CPU to periodically check the status of the Bluetooth connection, ensuring system stability without overwhelming the processor.
- **Serial Buffer Management:** To prevent **Data Loss**, the system utilizes a **Serial Receive Buffer**. The microcontroller reads the incoming bytes immediately from the buffer, ensuring that the control sequence remains synchronized with the user's inputs.
- **Power & Resource Efficiency:** Using a **Switch-Case** structure instead of multiple nested **If-Else** statements optimizes the **Program Counter (PC)** path, reducing the overall execution time for each instruction cycle.

9. Challenges & Engineering Solutions

- **Voltage Drops:** The motors caused the Arduino to reboot. **Solution:** Separating the power rails and using a common ground (GND).

- **Calibration:** Aligning the two servos to move in sync. **Solution:** Fine-tuning the `servo.write()` angles in the setup function.
- **Code Blocking:** The 4-second delay in the garage code stops other processes. **Solution:** Future iterations will replace `delay()` with the `millis()` function.

10. Conclusion

The **Smart Garage & 4WD Car** project successfully demonstrates the integration of hardware and software. By combining automated sensors with wireless control, we created a functional prototype for future smart-home systems